```
# 🧊 HVAC Optimization in Labs (Week 5)
#Decision Tree Cooling Forecast + Zone Heatmap Dashboard
'''
This notebook demonstrates:
- **Synthetic data generation** for lab occupancy, temperature, and humidity
- **Decision Tree model** to predict cooling demand (kW)
- **Interactive dashboard** with zone-wise heatmaps and diagnostics

**Goal:** Forecast cooling needs based on occupancy + environmental conditions to optimize HVAC energy usage.'''
```

```
'\nThis notebook demonstrates:\n- **Synthetic data generation** for lab occupancy, temperature, and humidity\n- **Decision Tree model** to predict cooling demand (kW)\n- **Interactive dashboard** with zone-wise heatmaps and diagnostics\n\n**Goal:** Forecast cooling needs based on occupancy + environmental conditions to optimize HVAC energy usage.'
```

```
# Import required libraries
import numpy as np
import pandas as pd
from datetime import datetime, timedelta

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

np.random.seed(42)
print(" Libraries imported")
```

```
 Libraries imported
```

📊 Step 1: Generate Synthetic Lab HVAC Data

We simulate multiple lab zones with occupancy, temperature, humidity, and resulting cooling demand (kW).

```
# Generate synthetic HVAC data
zones = ["Lab-A", "Lab-B", "Lab-C", "Lab-D", "Lab-E", "Lab-F"]

n_days = 60
hours = list(range(8, 21))  # 8 AM to 8 PM
start_date = datetime(2025, 9, 1)

records = []
for day in range(n_days):
    current_date = start_date + timedelta(days=day)
    day_of_week = current_date.weekday()
    is_weekend = day_of_week >= 5

    for hour in hours:
        # External temperature (seasonal + daily pattern)
        base_temp = 28 + 6 * np.sin(2 * np.pi * day / n_days)
        hour_temp = base_temp + 4 * np.sin(np.pi * (hour - 8) / 12) + np.random.normal(0, 1.5)

        # Humidity pattern
        humidity = 50 + 15 * np.sin(2 * np.pi * hour / 24) + np.random.normal(0, 5)
        humidity = np.clip(humidity, 30, 85)

        for zone in zones:
            # Occupancy pattern (higher during class hours)
            base_occ = 5 if is_weekend else 20
            peak_occ = 40 if 10 <= hour <= 16 else 15
            occupancy = base_occ + peak_occ + np.random.normal(0, 6)
            occupancy = int(max(0, occupancy))

            # Zone-specific offsets
            zone_temp_offset = {"Lab-A": 0, "Lab-B": 1.5, "Lab-C": -1, "Lab-D": 2, "Lab-E": -1.5, "Lab-F": 0.5}[zone]
            zone_occ_factor = {"Lab-A": 1.0, "Lab-B": 1.1, "Lab-C": 0.9, "Lab-D": 1.2, "Lab-E": 0.85, "Lab-F": 1.05}[zone]

            zone_temp = hour_temp + zone_temp_offset
            zone_occ = int(occupancy * zone_occ_factor)

            # Cooling demand (kW) depends on occupancy, temp, humidity
            cooling_kw = (
                12 + 0.7 * zone_occ + 1.1 * (zone_temp - 22) + 0.08 * (humidity - 40)
                + np.random.normal(0, 3)
            )
```

```
            cooling_kw = max(5, cooling_kw)

            records.append({
                "datetime": current_date.replace(hour=hour),
                "date": current_date.date(),
                "hour": hour,
                "day_of_week": day_of_week,
                "is_weekend": int(is_weekend),
                "zone": zone,
                "occupancy": zone_occ,
                "temperature": round(zone_temp, 1),
                "humidity": round(humidity, 1),
                "cooling_kw": round(cooling_kw, 2)
            })

df = pd.DataFrame(records)
print(f" Generated {len(df)} records")
print(df.head())
```

```
 Generated 4680 records
             datetime        date  hour  day_of_week  is_weekend   zone  \
0 2025-09-01 08:00:00  2025-09-01     8            0           0  Lab-A
1 2025-09-01 08:00:00  2025-09-01     8            0           0  Lab-B
2 2025-09-01 08:00:00  2025-09-01     8            0           0  Lab-C
3 2025-09-01 08:00:00  2025-09-01     8            0           0  Lab-D
4 2025-09-01 08:00:00  2025-09-01     8            0           0  Lab-E

   occupancy  temperature  humidity  cooling_kw
0         38         28.7      62.3       52.37
1         36         30.2      62.3       47.35
2         39         27.7      62.3       49.71
3         38         30.7      62.3       51.63
4         27         27.2      62.3       37.06
```

🧠 Step 2: Train Decision Tree Model

We train a basic decision tree using occupancy, temperature, humidity, hour, day, and weekend indicators to forecast cooling demand (kW).

```
# Prepare features and train decision tree
feature_cols = ["hour", "day_of_week", "is_weekend", "occupancy", "temperature", "humidity"]
X = df[feature_cols]
y = df["cooling_kw"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = DecisionTreeRegressor(
    max_depth=6,
    min_samples_leaf=15,
    random_state=42
)
model.fit(X_train, y_train)

# Evaluate model
train_pred = model.predict(X_train)
test_pred = model.predict(X_test)

print(" Decision Tree Performance")
print("=" * 50)
print(f"Train R²: {r2_score(y_train, train_pred):.4f}")
print(f"Test  R²: {r2_score(y_test, test_pred):.4f}")
print(f"Test RMSE: {np.sqrt(mean_squared_error(y_test, test_pred)):.2f} kW")
print(f"Test MAE: {mean_absolute_error(y_test, test_pred):.2f} kW")
```

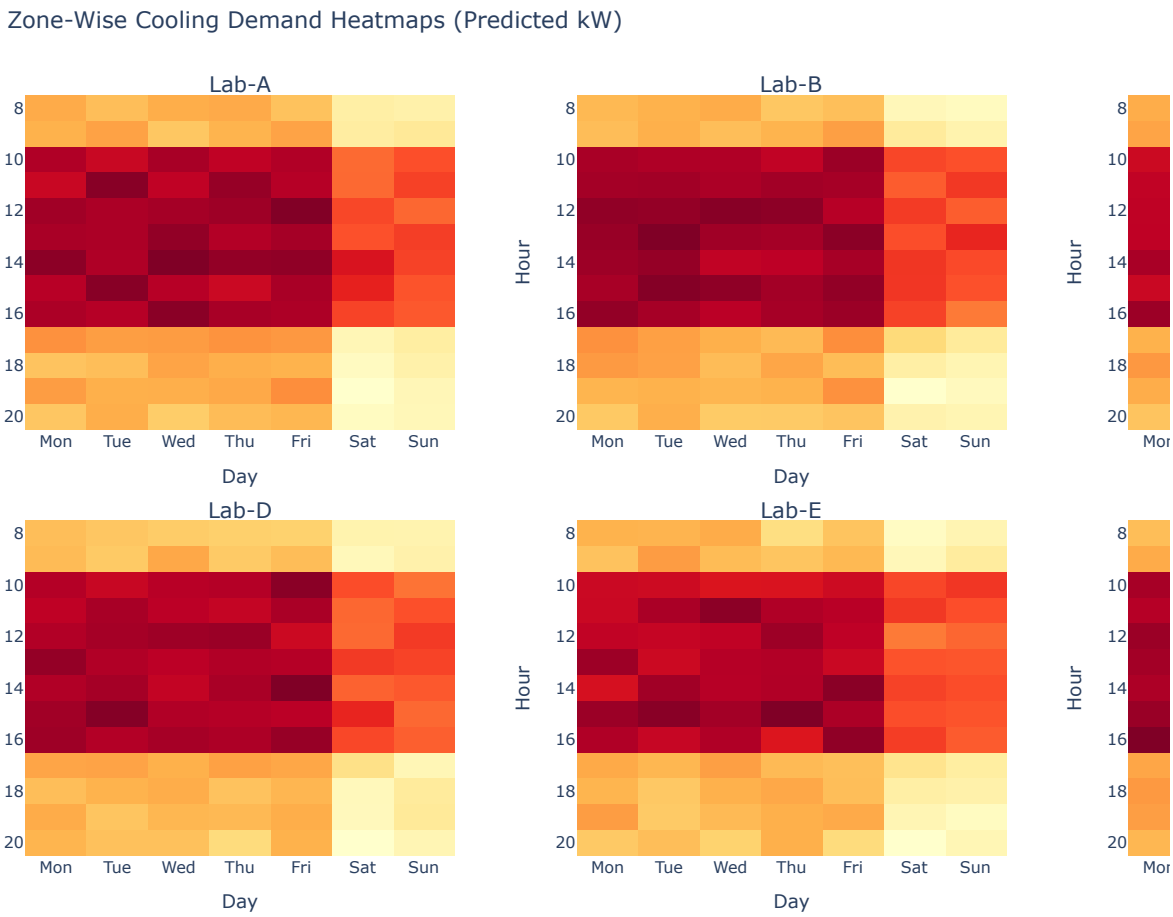
```
 Decision Tree Performance
==================================================
Train R²: 0.9437
Test  R²: 0.9340
Test RMSE: 3.72 kW
Test MAE: 2.95 kW
```

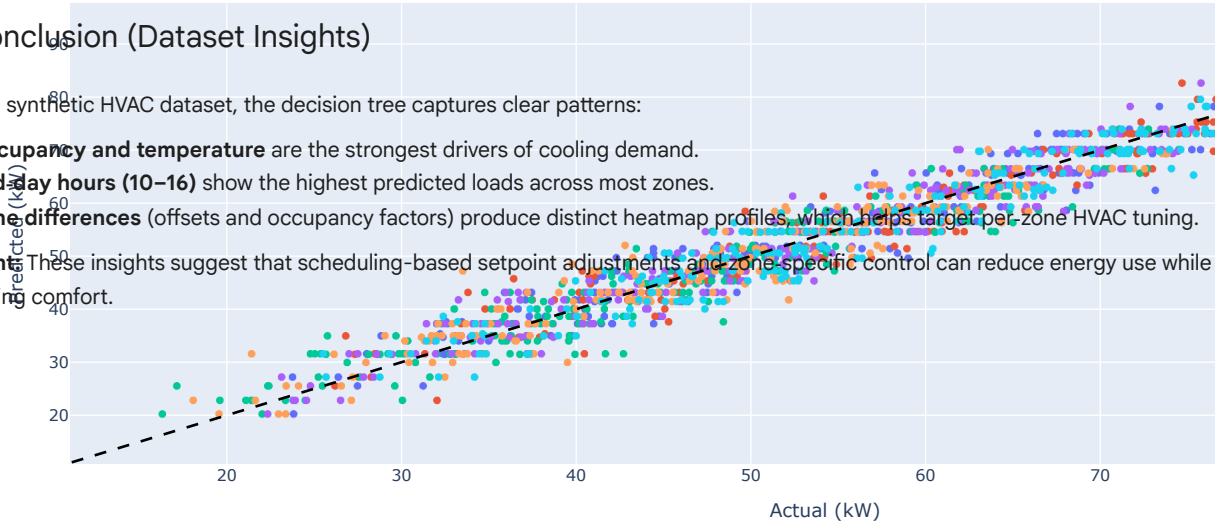
italicized text## 🧩 Step 3: Zone-Wise Heatmap Dashboard

We generate **zone-wise heatmaps** of predicted cooling demand across hours and days.

```
# Predict cooling demand for all records
pred_df = df.copy()
pred_df["predicted_kw"] = model.predict(pred_df[feature_cols])
```

```
# Dashboard: zone-wise heatmaps (hour vs day of week)
fig = make_subplots(
    rows=2, cols=3,
    subplot_titles=[f"{z}" for z in zones],
    vertical_spacing=0.12,
    horizontal_spacing=0.08
)

zone_positions = {
    "Lab-A": (1, 1), "Lab-B": (1, 2), "Lab-C": (1, 3),
    "Lab-D": (2, 1), "Lab-E": (2, 2), "Lab-F": (2, 3)
}

for zone in zones:
    z_df = pred_df[pred_df["zone"] == zone]
    pivot = z_df.pivot_table(
        values="predicted_kw",
        index="hour",
        columns="day_of_week",
        aggfunc="mean"
    )
    r, c = zone_positions[zone]
    fig.add_trace(
        go.Heatmap(
            z=pivot.values,
            x=["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"],
            y=pivot.index,
            colorscale="YlOrRd",
            colorbar=dict(title="kW") if zone == "Lab-C" else None
        ),
        row=r, col=c
    )

fig.update_layout(
    height=750,
    title_text=" Zone-Wise Cooling Demand Heatmaps (Predicted kW)",
    showlegend=False
)
fig.update_yaxes(title_text="Hour", autorange="reversed")
fig.update_xaxes(title_text="Day")
fig.show()

# Diagnostic: actual vs predicted scatter
scatter_fig = px.scatter(
    pred_df.sample(1500, random_state=42),
    x="cooling_kw",
    y="predicted_kw",
    color="zone",
    title=" Actual vs Predicted Cooling Demand",
    labels={"cooling_kw": "Actual (kW)", "predicted_kw": "Predicted (kW)"}
)
scatter_fig.add_shape(
    type="line",
    x0=pred_df["cooling_kw"].min(), y0=pred_df["cooling_kw"].min(),
    x1=pred_df["cooling_kw"].max(), y1=pred_df["cooling_kw"].max(),
    line=dict(color="black", dash="dash")
)
scatter_fig.show()
```





✅ Conclusion (Dataset Insights)

From the synthetic HVAC dataset, the decision tree captures clear patterns:

- **Occupancy and temperature** are the strongest drivers of cooling demand.
- **Mid-day hours (10–16)** show the highest predicted loads across most zones.
- **Zone differences** (offsets and occupancy factors) produce distinct heatmap profiles, which helps target per-zone HVAC tuning.

Comment: These insights suggest that scheduling-based setpoint adjustments and zone-specific control can reduce energy use while maintaining comfort.